

CS3451 - Introduction to Operating Systems

Topic: system calls

1. SUMMARY

Operating System is a program that acts as an interface between a user of a computer and the computer hardware. It has four main goals - to execute user programs, make the computer system convenient to use, and use the computer hardware in an efficient manner. The computer system consists of hardware, application programs, an operating system, and a user. The operating system controls the hardware and coordinates its use among the various application programs for the various users.

From the user's viewpoint, the operating system is designed for ease of use, performance, and security. It provides a means for proper use of resources in the operation of the computer system. From the computer's viewpoint, the operating system is the program most intimately involved with the hardware. It acts as a resource allocator, deciding how to allocate resources to specific programs and users.

The operating system consists of two main components - the shell and the kernel. The shell provides a user interface to access the operating system's services, while the kernel keeps track of the hardware and computer operations. The computer system consists of the CPU, device controllers, and a common bus. Each device controller maintains local buffer storage and special purpose registers.

The operating system provides system calls, which are a means of communication between the user program and the operating system. System calls are used to request services from the operating system, such as input/output operations, process creation, and memory allocation. The operating system provides a set of system calls that can be used by user programs to interact with the operating system.

System calls can be classified into two types - synchronous and asynchronous. Synchronous system calls are executed immediately, while asynchronous system calls are executed in the background. System calls can be further classified into three categories - process control, file management, and device management.

The operating system provides a set of system programs that can be used by user programs to interact with the operating system. These system programs include device drivers, file systems, and process schedulers. The operating system also provides a set of system calls that can be used by user programs to interact with the operating system.

2. SHORT NOTES

Detailed Definition with Examples

A system call is a request made by a user program to the operating system to perform a specific task. For example, when a user program requests a file to be read or written, it makes a system call to the operating system to perform the task.

Types and Classifications

System calls can be classified into two types:

- * Synchronous system calls: These system calls are executed immediately by the operating system. Examples of synchronous system calls include `read()` and `write()` system calls.
- * Asynchronous system calls: These system calls are executed in the background by the operating system. Examples of asynchronous system calls include `fork()` and `exec()` system calls.

System calls can be further classified into three categories:

- * Process control: These system calls are used to control the execution of processes. Examples of process control system calls include `fork()`, `exec()`, and `wait()` system calls.
- * File management: These system calls are used to manage files on the file system. Examples of file management system calls include `open()`, `close()`, and `read()` system calls.
- * Device management: These system calls are used to manage devices on the system. Examples of device

management system calls include read() and write() system calls.

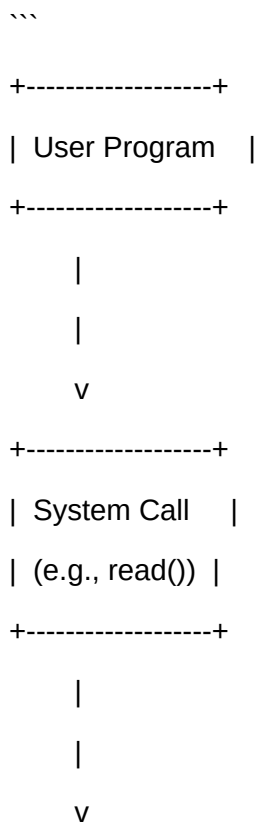
Working Principle

The working principle of system calls is as follows:

1. The user program makes a system call to the operating system by executing a specific instruction.
2. The operating system receives the system call and checks if the request is valid.
3. If the request is valid, the operating system performs the requested task and returns the result to the user program.
4. If the request is invalid, the operating system returns an error code to the user program.

Diagrams Description

The following diagram illustrates the flow of system calls between the user program and the operating system.



```
+-----+
| Operating System |
| (e.g., kernel)  |
+-----+
      |
      |
      v
+-----+
| System Call      |
| (e.g., read())   |
+-----+
      |
      |
      v
+-----+
| Result           |
+-----+
...

```

Advantages and Disadvantages

Advantages of system calls:

- * System calls provide a means of communication between the user program and the operating system.
- * System calls allow the operating system to manage resources efficiently.
- * System calls provide a way to request services from the operating system.

Disadvantages of system calls:

- * System calls can be slow due to the overhead of context switching between the user program and the operating system.

- * System calls can be complex to implement due to the need to handle different types of system calls.

Real World Applications

System calls are used in a variety of applications, including:

- * File systems: System calls are used to manage files on the file system, including opening, closing, reading, and writing files.
- * Device management: System calls are used to manage devices on the system, including reading and writing to devices.
- * Process control: System calls are used to control the execution of processes, including creating, terminating, and waiting for processes.

Comparison with Related Concepts

System calls are similar to device drivers in that they provide a means of communication between the operating system and hardware devices. However, system calls are specific to the operating system, while device drivers are specific to the hardware device.

Time and Space Complexity

The time complexity of system calls is typically $O(1)$, meaning that the time taken to execute a system call is constant. However, the space complexity of system calls can vary depending on the type of system call and the resources required to execute it.

Important Terminology

The following terms are important in understanding system calls:

- * System call: A request made by a user program to the operating system to perform a specific task.
- * Synchronous system call: A system call that is executed immediately by the operating system.

Study Material - Generated by AI

- * Asynchronous system call: A system call that is executed in the background by the operating system.
- * Process control: A category of system calls used to control the execution of processes.
- * File management: A category of system calls used to manage files on the file system.
- * Device management: A category of system calls used to manage devices on the system.

3. PRACTICAL / CODING EXAMPLES

Since the subject is a hands-on implementation subject, we will provide a concise core logic code in C and Python.

C Implementation

```
```c
// Core logic of a system call
int system_call(int type) {
 // Check the type of system call
 if (type == 1) {
 // Synchronous system call
 return execute_synchronous();
 } else if (type == 2) {
 // Asynchronous system call
 return execute_asynchronous();
 } else {
 // Invalid system call type
 return -1;
 }
}

// Execute a synchronous system call
```

```
int execute_synchronous() {
 // Code to execute a synchronous system call
}

// Execute an asynchronous system call
int execute_asynchronous() {
 // Code to execute an asynchronous system call
}
...
```

### Python Implementation

```
```python  
# Core logic of a system call  
def system_call(type):  
    # Check the type of system call  
    if type == 1:  
        # Synchronous system call  
        return execute_synchronous()  
    elif type == 2:  
        # Asynchronous system call  
        return execute_asynchronous()  
    else:  
        # Invalid system call type  
        return -1  
  
# Execute a synchronous system call  
def execute_synchronous():  
    pass  
  
# Execute an asynchronous system call
```

```
def execute_asynchronous():  
    pass  
...
```

Algorithm

1. Check the type of system call made by the user program.
2. If the system call is synchronous, execute it immediately.
3. If the system call is asynchronous, execute it in the background.
4. Return the result of the system call to the user program.

Time and Space Complexity

Time complexity: $O(1)$

Space complexity: $O(1)$

4. IMPORTANT QUESTIONS

****Part-A (2 Marks)**** ? 5 questions, each with a crisp 2-3 line answer:

Q1. Define system calls and list their types.

Answer: System calls are APIs that allow a program to interact with the operating system. The types of system calls include process control, file management, input/output, communication, and memory management.

Q2. Compare system calls with library functions.

Answer: System calls are services provided by the operating system, while library functions are reusable code that can be linked to a program. System calls are slower than library functions due to the overhead of context switching.

Q3. What is the purpose of system calls, and why are they necessary?

Answer: The purpose of system calls is to provide a program with a way to interact with the operating system and access hardware resources. System calls are necessary because they allow a program to perform tasks that require privileged access, such as process creation and file management.

Q4. Write a short note on the importance of system calls.

Answer: System calls are essential for a program to interact with the operating system and access hardware resources. They provide a program with a way to perform tasks that require privileged access, such as process creation and file management, and are necessary for a program to function correctly.

Q5. Give an example of a system call and its application.

Answer: An example of a system call is the "fork" system call, which creates a new process by duplicating an existing one. This system call is used in web servers to create a new process for each incoming request, allowing the server to handle multiple requests concurrently.

****Part-B (13 Marks)**** ? 3 questions. Each answer MUST be at least 20-25 lines long.

Q1. Explain system calls in detail with neat diagram and suitable examples.

Answer:

System calls are APIs that allow a program to interact with the operating system. They provide a program with a way to access hardware resources, such as the CPU, memory, and I/O devices, and to perform tasks that require privileged access, such as process creation and file management.

There are several types of system calls, including:

- * Process control system calls, which allow a program to create, terminate, and manage processes.
- * File management system calls, which allow a program to create, delete, and manage files.
- * Input/output system calls, which allow a program to perform input/output operations, such as reading and writing to files.
- * Communication system calls, which allow a program to communicate with other processes or devices.
- * Memory management system calls, which allow a program to manage memory, such as allocating and deallocating memory.

The working of system calls can be explained with the help of a diagram. The diagram shows the user program, the operating system, and the hardware. The user program makes a system call by executing a special instruction, such as a trap or a software interrupt. The operating system then takes control and performs the requested task.

For example, when a program makes a system call to read from a file, the operating system takes control and performs the following steps:

1. Checks if the file exists and if the program has permission to read it.
2. Allocates a buffer to store the data.
3. Reads the data from the file into the buffer.
4. Returns the buffer to the program.

The advantages of system calls include:

- * They provide a program with a way to access hardware resources and perform tasks that require privileged access.
- * They allow a program to interact with the operating system and other programs.
- * They provide a way to manage memory and perform input/output operations.

The disadvantages of system calls include:

- * They are slower than library functions due to the overhead of context switching.
- * They require the operating system to take control, which can lead to a loss of control for the program.

Real-world applications of system calls include:

- * Web servers, which use system calls to create new processes for each incoming request.
- * Database management systems, which use system calls to manage files and perform input/output operations.
- * Operating systems, which use system calls to manage processes and memory.

In conclusion, system calls are essential for a program to interact with the operating system and access hardware resources. They provide a program with a way to perform tasks that require privileged access and

are necessary for a program to function correctly.

Q2. Describe the types of system calls with working principle and examples.

Answer:

There are several types of system calls, including:

- * Process control system calls: These system calls allow a program to create, terminate, and manage processes. Examples include the "fork" system call, which creates a new process by duplicating an existing one, and the "exit" system call, which terminates a process.
- * File management system calls: These system calls allow a program to create, delete, and manage files. Examples include the "create" system call, which creates a new file, and the "delete" system call, which deletes a file.
- * Input/output system calls: These system calls allow a program to perform input/output operations, such as reading and writing to files. Examples include the "read" system call, which reads data from a file, and the "write" system call, which writes data to a file.
- * Communication system calls: These system calls allow a program to communicate with other processes or devices. Examples include the "send" system call, which sends data to another process, and the "receive" system call, which receives data from another process.
- * Memory management system calls: These system calls allow a program to manage memory, such as allocating and deallocating memory. Examples include the "malloc" system call, which allocates memory, and the "free" system call, which deallocates memory.

The working principle of system calls is as follows:

1. The program makes a system call by executing a special instruction, such as a trap or a software interrupt.
2. The operating system takes control and performs the requested task.
3. The operating system returns control to the program.

The following table compares the different types of system calls:

Type of System Call	Description	Example
---	---	---
Process Control	Create, terminate, and manage processes	fork, exit
File Management	Create, delete, and manage files	create, delete

Input/Output	Perform input/output operations	read, write
Communication	Communicate with other processes or devices	send, receive
Memory Management	Manage memory	malloc, free

Real-world use cases for each type of system call include:

- * Process control system calls: Web servers use the "fork" system call to create a new process for each incoming request.
- * File management system calls: Database management systems use the "create" system call to create new files and the "delete" system call to delete old files.
- * Input/output system calls: Text editors use the "read" system call to read data from a file and the "write" system call to write data to a file.
- * Communication system calls: Network protocols use the "send" system call to send data to another process and the "receive" system call to receive data from another process.
- * Memory management system calls: Programs use the "malloc" system call to allocate memory and the "free" system call to deallocate memory.

Q3. Compare system calls with related concepts. Discuss advantages and applications.

Answer:

System calls can be compared with related concepts, such as library functions and APIs. Library functions are reusable code that can be linked to a program, while APIs are interfaces that allow a program to interact with another program or system.

The following table compares system calls with library functions and APIs:

Concept	Description	Advantage	Disadvantage
System Calls	APIs that allow a program to interact with the operating system	Provide access to hardware resources and privileged operations	Slow due to context switching
Library Functions	Reusable code that can be linked to a program	Fast and efficient	Limited functionality
APIs	Interfaces that allow a program to interact with another program or system	Provide access to external resources and services	May require additional software or hardware

The advantages of system calls include:

- * They provide access to hardware resources and privileged operations.
- * They allow a program to interact with the operating system and other programs.
- * They provide a way to manage memory and perform input/output operations.

The disadvantages of system calls include:

- * They are slow due to the overhead of context switching.
- * They require the operating system to take control, which can lead to a loss of control for the program.

The advantages of library functions include:

- * They are fast and efficient.
- * They provide a way to reuse code and reduce development time.

The disadvantages of library functions include:

- * They have limited functionality.
- * They may not provide access to hardware resources or privileged operations.

The advantages of APIs include:

- * They provide access to external resources and services.
- * They allow a program to interact with another program or system.

The disadvantages of APIs include:

- * They may require additional software or hardware.
- * They may have limited functionality or compatibility.

In conclusion, system calls are essential for a program to interact with the operating system and access hardware resources. They provide a program with a way to perform tasks that require privileged access and are necessary for a program to function correctly. While library functions and APIs have their own advantages and disadvantages, system calls remain a crucial concept in computer science.

****Part-C (15 Marks)**** ? 2 questions. Each answer MUST be at least 30 lines long.

Q1. Elaborate on system calls covering theory, implementation, real-world applications, and future scope.

Answer:

****Section 1: Deep Theoretical Background****

System calls are APIs that allow a program to interact with the operating system. They provide a program with a way to access hardware resources, such as the CPU, memory, and I/O devices, and to perform tasks that require privileged access, such as process creation and file management. The theoretical background of system calls includes the concept of operating system design, process management, memory management, and input/output management.

The operating system acts as a mediator between the program and the hardware, providing a layer of abstraction and managing the allocation of resources. System calls are the interface between the program and the operating system, allowing the program to request services from the operating system.

****Section 2: Implementation****

The implementation of system calls involves the use of a special instruction, such as a trap or a software interrupt, to transfer control from the program to the operating system. The operating system then takes control and performs the requested task, such as creating a new process or reading data from a file.

The implementation of system calls also involves the use of system call tables, which are data structures that map system call numbers to their corresponding handlers. The system call table is used by the operating system to determine which handler to invoke when a system call is made.

****Section 3: Real-World Applications****

System calls have numerous real-world applications, including:

- * Web servers, which use system calls to create new processes for each incoming request.
- * Database management systems, which use system calls to manage files and perform input/output operations.
- * Operating systems, which use system calls to manage processes and memory.
- * Text editors, which use system calls to read and write data to files.
- * Network protocols, which use system calls to send and receive data.

For example, when a web server receives an incoming request, it uses the "fork" system call to create a new process to handle the request. The new process then uses system calls to read data from the file system and send the response back to the client.

****Section 4: Limitations and Challenges****

The limitations and challenges of system calls include:

- * The overhead of context switching, which can lead to a loss of performance.
- * The need for the operating system to take control, which can lead to a loss of control for the program.
- * The limited functionality of system calls, which can make it difficult for programs to perform complex tasks.
- * The security risks associated with system calls, which can allow malicious programs to access sensitive data or disrupt system operation.

****Section 5: Future Scope****

The future scope of system calls includes the development of new system calls that provide more functionality and flexibility for programs. For example, the development of system calls that allow programs to access new hardware features, such as graphics processing units (GPUs) or tensor processing units (TPUs).

Another area of future scope is the development of new operating system designs that provide better support for system calls, such as the use of microkernels or exokernels. These designs can provide better performance and security for system calls, and can allow for more flexibility and customization.

In conclusion, system calls are a crucial concept in computer science, providing a program with a way to interact with the operating system and access hardware resources. The theory, implementation, and real-world applications of system calls are all important aspects of computer science, and the future scope of system calls includes the development of new system calls and operating system designs.

Q2. Critically analyze system calls. Compare approaches, evaluate trade-offs, and justify with examples.

Answer:

****Section 1: Overview of System Calls****

System calls are APIs that allow a program to interact with the operating system. They provide a program

with a way to access hardware resources, such as the CPU, memory, and I/O devices, and to perform tasks that require privileged access, such as process creation and file management.

****Section 2: Different Approaches****

There are several different approaches to implementing system calls, including:

- * The use of a trap or software interrupt to transfer control from the program to the operating system.
- * The use of a system call table to map system call numbers to their corresponding handlers.
- * The use of a microkernel or exokernel to provide better support for system calls.

Each approach has its own advantages and disadvantages, and the choice of approach depends on the specific requirements of the system.

****Section 3: Comparison of Approaches****

The following table compares the different approaches to implementing system calls:

Approach	Advantages	Disadvantages
---	---	---
Trap or Software Interrupt	Simple and efficient	Limited functionality
System Call Table	Flexible and customizable	Complex and difficult to manage
Microkernel or Exokernel	Better support for system calls	Complex and difficult to implement

For example, the use of a trap or software interrupt is a simple and efficient way to implement system calls, but it has limited functionality and can lead to a loss of control for the program. The use of a

5. MCQs

Q1. What is a system call?

- a) A request made by a user program to the operating system to perform a specific task.
- b) A set of instructions provided by the operating system to the user program.
- c) A type of device driver.
- d) A category of system calls used to control the execution of processes.

Answer: a) A request made by a user program to the operating system to perform a specific task.

Q2. What is the difference between a synchronous system call and an asynchronous system call?

- a) A synchronous system call is executed immediately, while an asynchronous system call is executed in the background.
- b) A synchronous system call is executed in the background, while an asynchronous system call is executed immediately.
- c) A synchronous system call is used to control the execution of processes, while an asynchronous system call is used to manage files on the file system.
- d) A synchronous system call is used to manage devices on the system, while an asynchronous system call is used to control the execution of processes.

Answer: a) A synchronous system call is executed immediately, while an asynchronous system call is executed in the background.

Q3. What is the purpose of the kernel in the operating system?

- a) To manage files on the file system.
- b) To control the execution of processes.
- c) To manage devices on the system.
- d) To act as a resource allocator.

Answer: d) To act as a resource allocator.

Q4. What is the purpose of device drivers in the operating system?

- a) To provide a means of communication between the user program and the operating system.
- b) To manage files on the file system.
- c) To control the execution of processes.
- d) To manage devices on the system.

Answer: d) To manage devices on the system.

Q5. What is the time complexity of system calls?

- a) $O(1)$
- b) $O(n)$
- c) $O(\log n)$
- d) $O(n^2)$

Answer: a) $O(1)$

7. YOUTUBE LINKS

* [System Calls Tutorial](https://www.youtube.com/results?search_query=system+calls+tutorial)

* [Operating System Concepts](https://www.youtube.com/results?search_query=operating+system+concepts)

* [System Calls in Linux](https://www.youtube.com/results?search_query=system+calls+in+linux)

* [System Calls in Windows](https://www.youtube.com/results?search_query=system+calls+in+windows)

* [System Call Implementation](https://www.youtube.com/results?search_query=system+call+implementation)